

Introduction to Basic and Advanced Statistical Modelling

Making workflows reproducible

Alexandre Courtiol

Table of contents

What is reproducibility?

Why making workflows reproducible?

How to make workflows repeatable?

A quick introduction to quarto

Beyond repeatability

What is reproducibility?

Reproducibility is not binary

Reproducibility is best seen as a continuum within a multidimensional space:

- **repeatability**: can others obtain the same results given access to the same data?
- **transparency**: are explanations good enough to be understood by others?
- **accessibility**: can others find and download the exact data and code used?
- **compatibility**: can they run the code on a different computer/OS today?
- **sustainability**: will the code still run in X years after package updates?
- **generalisability**: can the code be applied to a new, similar dataset?

Reproducibility is not binary

Reproducibility is best seen as a continuum within a multidimensional space:

- **repeatability**: can others obtain the same results given access to the same data?
- **transparency**: are explanations good enough to be understood by others?
- **accessibility**: can others find and download the exact data and code used?
- **compatibility**: can they run the code on a different computer/OS today?
- **sustainability**: will the code still run in X years after package updates?
- **generalisability**: can the code be applied to a new, similar dataset?

NB:

- names are my own (many synonymous exist)
- improving any dimension takes intentional time/effort and introduce trade-offs (e.g. improving generalisability decreases clarity)
- in this very short introduction, I focus on **repeatability**

Why making workflows reproducible?

Increasing reproducibility benefits people

Given that reproducibility comes at a cost, the main question to answer is:
who shall benefit from the effort of making your code reproducible?

Increasing reproducibility benefits people

Given that reproducibility comes at a cost, the main question to answer is:
who shall benefit from the effort of making your code reproducible?

- yourself
 - ☒ yourself right now
 - ☐ yourself in the near future (e.g. writing the paper)
 - ☐ yourself in the distant future (e.g. analysing similar data)

Increasing reproducibility benefits people

Given that reproducibility comes at a cost, the main question to answer is:
who shall benefit from the effort of making your code reproducible?

- yourself
 - ☒ yourself right now
 - ☐ yourself in the near future (e.g. writing the paper)
 - ☐ yourself in the distant future (e.g. analysing similar data)
- your immediate circle
 - ☐ your supervisor / PI
 - ☐ your colleagues and collaborators
 - ☐ potential reviewers, auditors, or evaluators

Increasing reproducibility benefits people

Given that reproducibility comes at a cost, the main question to answer is:
who shall benefit from the effort of making your code reproducible?

- yourself
 - ☒ yourself right now
 - ☐ yourself in the near future (e.g. writing the paper)
 - ☐ yourself in the distant future (e.g. analysing similar data)
- your immediate circle
 - ☐ your supervisor / PI
 - ☐ your colleagues and collaborators
 - ☐ potential reviewers, auditors, or evaluators
- the wider community
 - ☐ those interested in your data
 - ☐ those interested in your research question
 - ☐ those interested in your methodology

Increasing reproducibility benefits people

Given that reproducibility comes at a cost, the main question to answer is:
who shall benefit from the effort of making your code reproducible?

- yourself
 - ☒ yourself right now
 - ☐ yourself in the near future (e.g. writing the paper)
 - ☐ yourself in the distant future (e.g. analysing similar data)
- your immediate circle
 - ☐ your supervisor / PI
 - ☐ your colleagues and collaborators
 - ☐ potential reviewers, auditors, or evaluators
- the wider community
 - ☐ those interested in your data
 - ☐ those interested in your research question
 - ☐ those interested in your methodology

NB: many answers can simultaneously apply

Universal benefits of repeatable code

In comparison to just producing results on the fly (i.e. taken from the R console), writing repeatable scripts/documents provides the following benefits:

- it prevents coding mistakes
- exactly what you did is tracked¹
- updating the results after updating the data is easy

¹a repeatable code tracks what code does but not how the code changed over time; the latter can instead can be done using a version control system, such as git/GitHub

How to make workflows repeatable?

Various approach can be used to improve repeatability of R-based workflows

Tool	Literate programming**	Setup	Learning curve
R script	no	simple	low
R package*	yes	difficult	high
sweave	yes	medium	high
rmarkdown/quarto	yes	simple	low/mid

*i.e a specific R package which main goal is to reproduce the results of a workflow

**mixing prose and code

Various approach can be used to improve repeatability of R-based workflows

Tool	Literate programming**	Setup	Learning curve
R script	no	simple	low
R package*	yes	difficult	high
sweave	yes	medium	high
rmarkdown/quarto	yes	simple	low/mid

*i.e a specific R package which main goal is to reproduce the results of a workflow

**mixing prose and code

Conclusion:

- rmarkdown/quarto improves repeatability at low cost
- syntax errors in rmarkdown/quarto can be frustrating, but LLMs are exceptionally efficient at debugging these specific issues
- quarto can do the same & more than rmarkdown, so I will focus on that

A quick introduction to quarto

Practice 1

- create a new quarto HTML document using RStudio menu and render it

What is quarto?

It is defined as an “open-source scientific and technical publishing system”



- the R package {knitr} converts quarto files into the markdown format
- the software pandoc converts markdown files into various output formats

Key structure

1. a header defining the general properties of the document, in YAML format
2. body of text in markdown, html, or LaTeX
3. chunks of computer code

The header

Minimal example

```
---  
title: "Some title"  
format: html  
---
```

The header

A real example

```
---
title: "Introduction to Basic and Advanced Statistical Modelling"
subtitle: "Making workflows reproducible"
author: "Alexandre Courtiol"
format:
  beamer:
    aspectratio: 169
    toc: true
    classoption: t
header-includes: |
  \setbeamertemplate{frametitle continuation}{}
  \setbeamertemplate{itemize items}[circle]
  \usepackage{caption}
  \captionsetup{labelformat=empty, justification=centering}
  \setbeamertemplate{footline}{%
    \hfill\usebeamerfont{page number in head/foot}%
    \usebeamerfont{page number in head/foot}%
    \insertframenumber{} / \inserttotalframenumber\kern1em\vskip2pt%
  }
  \usepackage{xcolor}
  \let\oldtexttt\texttt
  \renewcommand{\texttt}[1]{\colorbox{gray!20}{\oldtexttt{#1}}}
editor_options:
  chunk_output_type: console
---
```

NB: this is part of the one used for this presentation

The header

The header is a particularly fragile component

- spacing matters
- copy/pasting headers across documents usually leads to unattended changes
- current version(s?) of RStudio sometimes delete lines from the headers automatically (bug)
- much of the content must be tailored to the selected output format

The body of text

This is the easy part

- write plain text and use `#`, `##`, `###` to define a hierarchy of sections (if the hierarchy is messed up it can be because a chunk is located at the highest level of the hierarchy)
- use `*text*`, `**text**`, `***text***`, to create *text*, **text** & ***text***, respectively
- use `[link_text](weblink)` to create hyperlinks
- use `![Figure legend](path_to_figure/fig.png){height=Xcm}` to add a figure of Xcm in height*

The body of text

This is the easy part

- write plain text and use `#`, `##`, `###` to define a hierarchy of sections (if the hierarchy is messed up it can be because a chunk is located at the highest level of the hierarchy)
- use `*text*`, `**text**`, `***text***`, to create *text*, **text** & ***text***, respectively
- use `[link_text](weblink)` to create hyperlinks
- use `![Figure legend](path_to_figure/fig.png){height=Xcm}` to add a figure of Xcm in height*

*to add figures, you can alternatively use HTML, LaTeX, or R commands. I prefer using the R command provided by `{knitr}` since it works well irrespective of the output format: inside a chunk of R code (see next slide), use `knitr::include_graphics("path_to_figure/fig.png")`; you can influence the formatting via chunk options (e.g. `#| out-height: Xcm`, `#| fig-align: center`)

Chunks of R code

```
```{r}  
#| option1: argument for option1
...
#| optionN: argument for optionN
some R code
```
```

Example

```
```{r}  
#| label: illustrating filter
#| echo: true
#| message: true
species_count |> filter(year == 2025)
```
```

Chunks of R code

```
```{r}  
#| option1: argument for option1
...
#| optionN: argument for optionN
some R code
```
```

Example

```
```{r}  
#| label: illustrating filter
#| echo: true
#| message: true
species_count |> filter(year == 2025)
```
```

Pro tips:

- defining labels using `#| label: some label` helps to identify where issues arise
- render the document frequently to discover mistakes soon after they are introduced

Practice 2

1. read the page quarto.org/docs/authoring/markdown-basics.html from the beginning until the end of the section entitled “Tables”
2. read the entire page quarto.org/docs/computations/r.html
3. add an R chunk, an image, and some text into your quarto document and render it
4. switch the format from `html` to `pdf` and render the document again²
5. explore the quarto guide: quarto.org/docs/guide

²this will require installing TinyTeX

Beyond repeatability

How to improve other aspects of reproducibility

- **transparency**: document *why* you do things and not just *how* you do things

How to improve other aspects of reproducibility

- **transparency**: document *why* you do things and not just *how* you do things
- **accessibility**: host your data and code on platforms such as GitHub, Zenodo, ...

How to improve other aspects of reproducibility

- **transparency**: document *why* you do things and not just *how* you do things
- **accessibility**: host your data and code on platforms such as GitHub, Zenodo, ...
- **compatibility**: make sure to try your code on other computers

How to improve other aspects of reproducibility

- **transparency**: document *why* you do things and not just *how* you do things
- **accessibility**: host your data and code on platforms such as GitHub, Zenodo, ...
- **compatibility**: make sure to try your code on other computers
- **sustainability**: minimize external dependencies; at minima document package version using `sessionInfo()` , or better use systems such as {renv} which save and load specific versions of R packages; for longer sustainability learn how to build containers (e.g. docker)

How to improve other aspects of reproducibility

- **transparency**: document *why* you do things and not just *how* you do things
- **accessibility**: host your data and code on platforms such as GitHub, Zenodo, ...
- **compatibility**: make sure to try your code on other computers
- **sustainability**: minimize external dependencies; at minima document package version using `sessionInfo()` , or better use systems such as {renv} which save and load specific versions of R packages; for longer sustainability learn how to build containers (e.g. docker)
- **generalisability**: design functions, do not hardcode parameters, do not use indices

How to improve other aspects of reproducibility

- **transparency**: document *why* you do things and not just *how* you do things
- **accessibility**: host your data and code on platforms such as GitHub, Zenodo, ...
- **compatibility**: make sure to try your code on other computers
- **sustainability**: minimize external dependencies; at minima document package version using `sessionInfo()`, or better use systems such as {renv} which save and load specific versions of R packages; for longer sustainability learn how to build containers (e.g. docker)
- **generalisability**: design functions, do not hardcode parameters, do not use indices

How to improve other aspects of reproducibility

- **transparency**: document *why* you do things and not just *how* you do things
- **accessibility**: host your data and code on platforms such as GitHub, Zenodo, ...
- **compatibility**: make sure to try your code on other computers
- **sustainability**: minimize external dependencies; at minima document package version using `sessionInfo()`, or better use systems such as {renv} which save and load specific versions of R packages; for longer sustainability learn how to build containers (e.g. docker)
- **generalisability**: design functions, do not hardcode parameters, do not use indices

NB:

- many of these improvements can be achieved at minimal costs
- remember that repeatability is a continuum (e.g. the more generalizable the code one wants, the more effort one must put into creating the code)